FULL STACK DEVELOPMENT LAB

B.Tech — Practical / Lab Manual

LAB PRACTICAL

Design and Development of a Full-Stack "Student Record Management" Web Application

Name: _____Shiv Bhatia_____

Roll No. / Enrollment No.: _____55_____

Class / Semester: _____CSE 'A' / 5th Sem_____

Duration: 2 Hours

Date: _____

1. Aim

To design, develop, and test a full-stack CRUD (Create, Read, Update, Delete) web application that integrates a frontend UI, a backend REST API, and a database, demonstrating the complete request-response cycle of a full-stack system.

2. Objectives

- To design and connect a frontend, backend, and database as a single working system.
- To build and test RESTful API endpoints using Express.js (or an equivalent backend framework).
- To perform CRUD operations on a database using an ORM/ODM (Mongoose / Sequelize).
- To consume REST APIs from the frontend using fetch/axios (AJAX-style communication).
- To apply client-side form validation and understand the client–server request-response cycle.
- To test APIs independently using Postman before frontend integration.

3. Software / Tools Required

- Frontend: HTML5, CSS3, JavaScript (or React.js)
- Backend: Node.js with Express.js
- Database: MongoDB (Mongoose) or MySQL (Sequelize)
- API Testing Tool: Postman / Thunder Client
- Code Editor: Visual Studio Code
- Browser: Google Chrome / Mozilla Firefox with Developer Tools

4. Problem Statement

Build a Student Record Management System with the following features:

- A form (frontend) to add a new student record: Name, Roll No., Course, Marks.
- A backend REST API exposing endpoints to add, view, update and delete student records.
- A database collection/table that persists the student data.
- A frontend page that displays all students in a table with Edit and Delete actions.
- Basic client-side form validation (e.g., Roll No. required, Marks between 0 and 100).
- Use of fetch/axios with async/await for all frontend–backend communication.

5. REST API Endpoints to be Implemented

HTTP Method	Endpoint	Description
POST	/students	Add a new student record
GET	/students	Fetch all student records
GET	/students/:id	Fetch a single student record by ID
PUT	/students/:id	Update an existing student record
DELETE	/students/:id	Delete a student record

Code

server.js

```
const express = require("express");
const mongoose = require("mongoose");


const app = express();


app.use(express.json());
app.use(express.static("public"));


mongoose.connect("mongodb://127.0.0.1:27017/studentDB");


const Student = mongoose.model("Student", {
    name: String,
    rollNo: String,
    course: String,
    marks: Number
});


// POST - Add
app.post("/students", async (req, res) => {
    const student = await Student.create(req.body);
    res.json({ message: "Student added", student });
});


// GET - All
app.get("/students", async (req, res) => {
```

```
    const students = await Student.find();
    res.json(students);
});

// GET - Single
app.get("/students/:id", async (req, res) => {
    const student = await Student.findById(req.params.id);
    res.json(student);
});

// PUT - Update
app.put("/students/:id", async (req, res) => {
    const student = await Student.findByIdAndUpdate(
        req.params.id,
        req.body,
        { new: true }
    );
    res.json({ message: "Student updated", student });
});

// DELETE
app.delete("/students/:id", async (req, res) => {
    await Student.findByIdAndDelete(req.params.id);
    res.json({ message: "Student deleted" });
});

app.listen(3000, () => {
    console.log("Server running on http://localhost:3000");
});
```

Output

```
Server running on http://localhost:3000
```

Frontend — public/index.html

```
<!DOCTYPE html>
```

```
<html>
<head>
    <title>Student Management</title>
</head>

<body>

<h1>Student Record Management</h1>

<form id="form">

    <input id="name" placeholder="Name" required>
    <input id="rollNo" placeholder="Roll No." required>
    <input id="course" placeholder="Course" required>

    <input
        id="marks"
        type="number"
        min="0"
        max="100"
        placeholder="Marks"
        required
    >

    <button>Add Student</button>

</form>

<table border="1">
    <thead>
        <tr>
            <th>Name</th>
            <th>Roll No.</th>
            <th>Course</th>
            <th>Marks</th>
```

```
        </tr>
    </thead>

    <tbody id="data"></tbody>
</table>

<script src="script.js"></script>

</body>
</html>
```

Output

```
Student Record Management

Name:     [ Shivanshi Sharma ]
Roll No.: [ 58 ]
Course:   [ CSE ]
Marks:    [ 92 ]

[ Add Student ]

-------------------------------------
Name             Roll No  Course Marks
Shivanshi Sharma  58       CSE    92
-------------------------------------
```

JavaScript — public/script.js

```
const form = document.getElementById("form");

form.addEventListener("submit", async function(e) {

    e.preventDefault();

    const marks = Number(document.getElementById("marks").value);
```

```
    if (marks < 0 || marks > 100) {
        alert("Marks must be between 0 and 100");
        return;
    }

    const student = {
        name: document.getElementById("name").value,
        rollNo: document.getElementById("rollNo").value,
        course: document.getElementById("course").value,
        marks: marks
    };

    await fetch("/students", {
        method: "POST",
        headers: {
            "Content-Type": "application/json"
        },
        body: JSON.stringify(student)
    });

    alert("Student added");
    loadStudents();
    form.reset();
});

async function loadStudents() {

    const response = await fetch("/students");
    const students = await response.json();

    document.getElementById("data").innerHTML =
        students.map(s => `
            <tr>
                <td>${s.name}</td>
                <td>${s.rollNo}</td>
```